Backend разработка

TBank X ВШЭ

Бородин Евгений, 10.04

Евгений

TeamLead в Т-Банке

- 6 лет в Java-разработке
- Разработчик в МФТИ, JetBrains, а затем и в Т-Банке
- Закончил КТ в ИТМО
- Обожаю путешествия, играю в футбол, а в этом модуле прочитаю оставшуюся часть курса по Backend-разработке

Архитектура приложения

План занятия

1. Правила игры
2. Что изучим до конца курса?
3. Паттерны при разработке:

   3.1.SOLID

   3.2. Многопоточное программирование

   3.3. Обработка ошибок

   3.4. Graceful Shutdown

   3.5. Кеширование

Итог = Округление(0.3 * ДЗ + 0.4 * ПР + 0.3 * Э)
Автомат = Округление((0.3 * ДЗ + 0.4 * ПР)/0.7)

где ДЗ — средняя оценка за все домашние задания (10 штук),
ПР — оценка за индивидуальный проект,
Э — оценка за экзамен.
Округление арифметическое.
ДЗ за первые 3 модуля можно прислать до 24 апреля включительно без потери баллов.

1. Правила игры

2. Что изучим до конца года?

(1) 10.04	**Правила игры + Архитектура приложения**
(2) 17.04	**Публикация приложения** Работа по сети — TLS, сертификаты, HTTPS. Запуск приложения на виртуальной машине в облаке.
(3) 24.04	**CI/CD (Gitlab)** Структура проекта. Разделение проекта на модули, Version Catalog, линтеры и code style. Контейнеризация, публикация артефактов в registry.
Нужно ли нам тут промежуточное занятие для сдачи домашек?	
(4) 15.05	**CI/CD(2)** Автоматизация тестирования, интеграционные тесты, testcontainers. Старт приложения в локальном окружении, Docker Compose.
(5) 22.05	**CI/CD(3)** Построение пайплайнов в Github (Gitlab), автоматизация сборки релиза и деплоя на виртуальную машину, секреты.
(6) 29.05	**Архитектура распределённых систем.** Асинхронное взаимодействие — batch processing, стриминг, очереди сообщений, гарантии доставки. Идемпотентность. Kafka и Spring.
(7) 05.06	**Архитектура распределённых систем** Партиционирование и репликация. Zero-downtime deployment: blue-green, red-black. Принципы работы Kubernetes.
(8) 19.06	**Сдача / защита домашек.** Последняя возможность получить баллы на автомат.

3.1 SOLID

Расшифровка:

- **S** — Single Responsibility Principle
- **O** — Open/Closed Principle
- **L** — Liskov Substitution Principle
- **I** — Interface Segregation Principle
- **D** — Dependency Inversion Principle

S — Single Responsibility Principle

Принцип единственной ответственности

У класса должна быть только одна причина для изменения.

Проще: Один класс — одна понятная задача.

Пример нарушения принципа

Пример соблюдения принципа

O — Open/Closed Principle

Принцип открытости / закрытости

Классы должны быть открыты для расширения, но закрыты для изменения.

Проще: Новое поведение лучше добавлять через новые классы, а не переписывать старые.

Пример нарушения принципа

Пример соблюдения принципа

L — Liskov Substitution Principle

Принцип подстановки Барбары Лисков

Объекты дочерних классов должны корректно заменять объекты родительских классов.

Проще: Если код работает с родительским типом, он должен нормально работать и с его наследниками.

Пример нарушения принципа

Пример соблюдения принципа

I — Interface Segregation Principle

Принцип разделения интерфейса

Классы-наследники не должны зависеть от методов, которые они не используют.

Проще: Лучше несколько маленьких интерфейсов, чем один огромный.

Пример нарушения принципа

Пример соблюдения принципа

D — Dependency Inversion Principle

Принцип инверсии зависимостей

Модули высокого уровня не должны зависеть от модулей низкого уровня.
И те, и другие должны зависеть от абстракций.

Проще: Зависим не от конкретного класса, а от интерфейса.

Пример нарушения принципа

Пример соблюдения принципа

Зачем нужен SOLID?

Наш код становится:

- легче читать,
- проще тестировать,
- удобнее расширять,
- безопаснее менять.